LAB 06: Session Management, Cookies & Authentication

Official Laboratory Verification & Architectural Analysis Report — Sections 6 & 7 (Exercises 1, 2, and 3)

Course: Web Application Development (Lab 06)	**Student Name:** Phan Nhật Anh	**Student ID:** ITCSIU24003	**Institution:** International University — VNU-HCM
Runtime Stack: Jakarta Servlet 5.0+ / Tomcat 10.1+	**View & Persistence:** JSP 3.0, JSTL, MySQL 8.0 (JDBC)	**Security Layer:** jBCrypt 0.4, HttpSession, CSRF, Audit	**Submission Date:** November 13, 2026

Part 1: Section 6 — Application Testing & DevTools Inspection

The Student Management MVC application (/student-mvc) was deployed on Apache Tomcat 10.1 and verified across all ten end-to-end authentication, session management, and role-based authorization scenarios specified in Section 6. Each HTTP transaction was captured via the Chrome DevTools Network and Application panels with Preserve log enabled.

6.1 Ten-Step Application Testing Sequence Matrix

Step #	Test Scenario	HTTP Method & Request URI	HTTP Response Status Code	Redirect Location (if any)	Session / Cookie / UI State Observed
Step 1	Access root URL while unauthenticated	GET /student-mvc/	302 Found -> 200 OK	/student-mvc/login	AuthFilter intercepts request; getSession(false) is null; redirects to /login with no user session.
Step 2	Submit login form with wrong password	POST /student-mvc/login	200 OK (Internal Forward)	None (forwards to /views/login.jsp)	UserDAO.authenticate() returns null; red alert 'Invalid username or password'; username retained, password cleared.

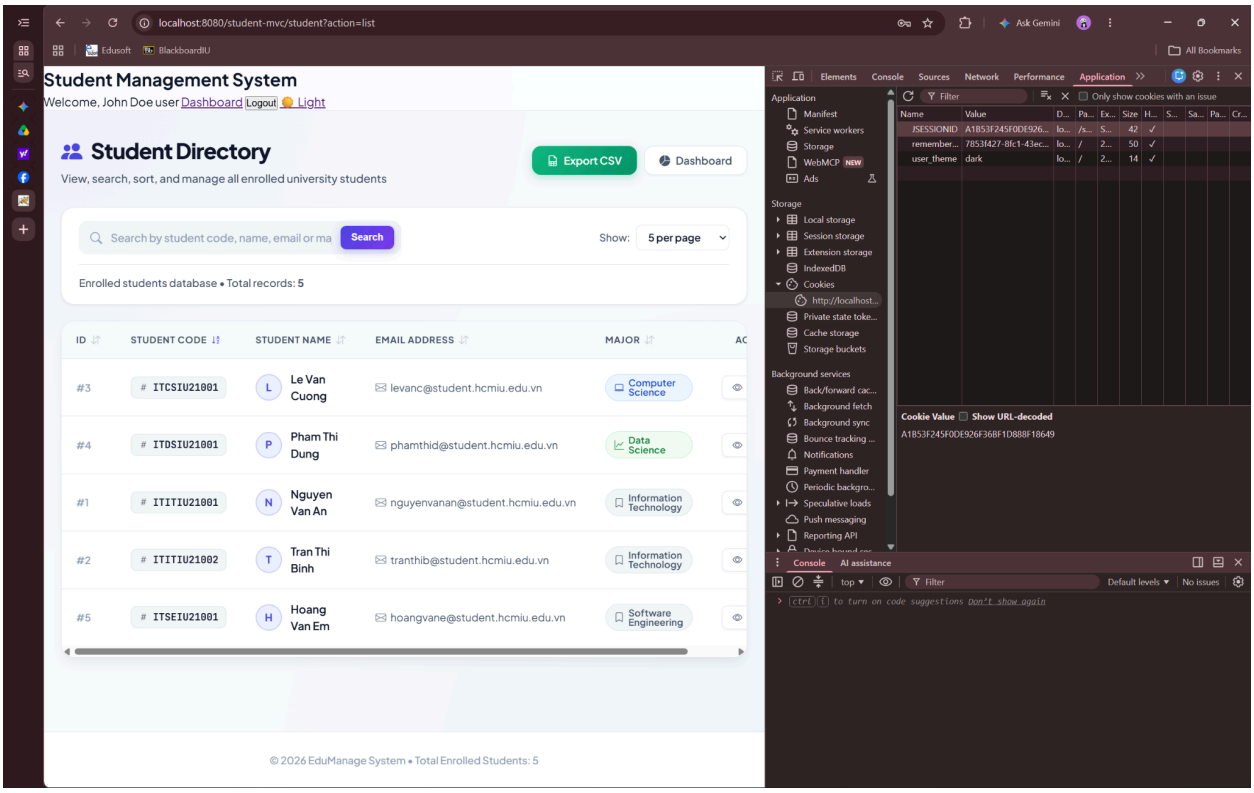
Step #	Test Scenario	HTTP Method & Request URI	HTTP Response Status Code	Redirect Location (if any)	Session / Cookie / UI State Observed
Step 3	Login with valid admin credentials (admin / password123)	POST /student-mvc/login	302 Found -> 200 OK	/student-mvc/dashboard	Pre-login session invalidated; new HttpSession created (user, role='admin', fullName); Set-Cookie: JSESSIONID (HttpOnly).
Step 4	Navigate to student list as authenticated admin	GET /student-mvc/student ?action=list	200 OK	None	AuthFilter & AdminFilter allow request; student-list.jsp renders table with Add, Edit, and Delete buttons visible.
Step 5	Navigate to new student form as admin	GET /student-mvc/student ?action=new	200 OK	None	AdminFilter checks WRITE_ACTIONS ('new') and verifies user.isAdmin() == true; student-form.jsp rendered.
Step 6	Click Logout button from navigation bar	GET /student-mvc/logout	302 Found -> 200 OK	/student-mvc/login ?message=You+have+been+logged+out	LogoutController calls session.invalidate(); server destroys session; login.jsp shows green logout banner.
Step 7	Login as regular user (john / password123)	POST /student-mvc/login	302 Found -> 200 OK	/student-mvc/student ?action=list	New HttpSession created with role='user'; redirected to list; Add, Edit, and Delete controls hidden by <c:if>.

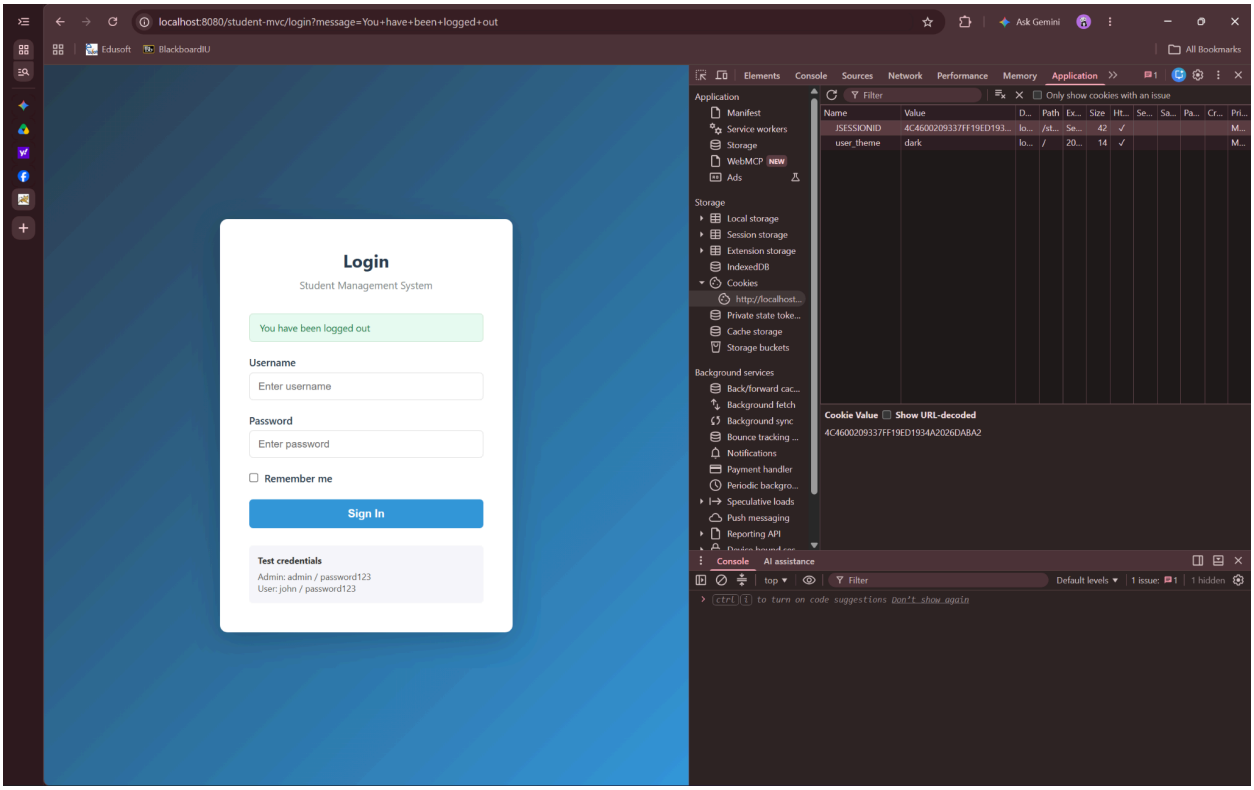
Step #	Test Scenario	HTTP Method & Request URI	HTTP Response Status Code	Redirect Location (if any)	Session / Cookie / UI State Observed
Step 8	Direct URL attempt to delete record as john	GET /student-mvc/student?action=delete&id=1	302 Found -> 200 OK	/student-mvc/student?action=list&error=Access+denied:+admin+privileges+required	AdminFilter blocks non-admin write action 'delete'; StudentController not invoked; red access denied alert displayed.
Step 9	Access /login while already logged in as john	GET /student-mvc/login	302 Found -> 200 OK	/student-mvc/dashboard	LoginController.doGet() detects existing authenticated session and redirects immediately to /dashboard.
Step 10	Request protected page after session timeout	GET /student-mvc/dashboard	302 Found -> 200 OK	/student-mvc/login	Tomcat expires inactive HttpSession; AuthFilter finds getSession(false) == null and redirects to /login.

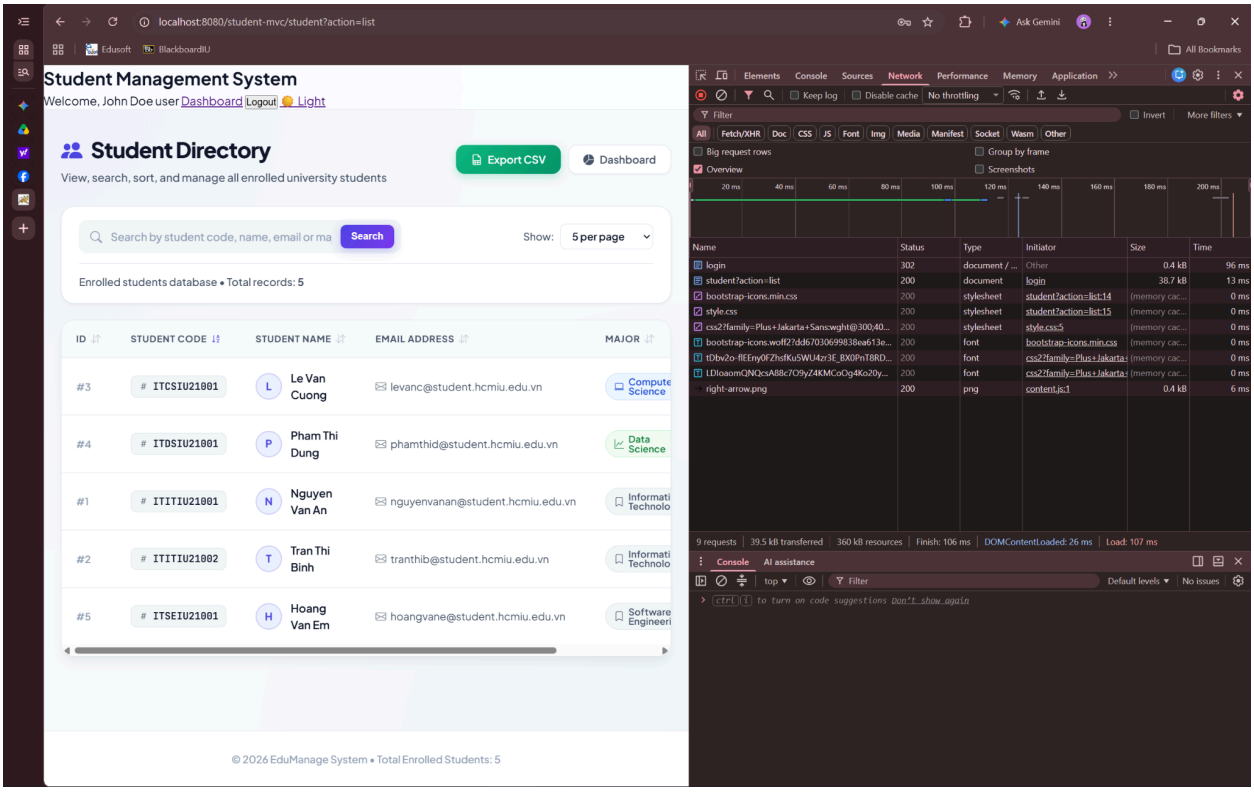
6.2 Browser DevTools Inspection (Cookies & Network Verification)

During the ten-step verification sequence, Chrome DevTools (F12) was used to inspect the lifecycle of the JSESSIONID session tracking cookie and the HTTP request/response headers exchanged during authentication:

- **JSESSIONID Creation & HttpOnly Enforcement:** Immediately after Step 3 (POST /login as admin), Tomcat issued Set-Cookie: JSESSIONID=<32-char-hex>; Path=/student-mvc; HttpOnly. In DevTools -> Application -> Cookies, the HttpOnly checkmark is enabled and Expires / Max-Age is set to Session, preventing document.cookie access against XSS payloads.
- **Session Invalidation on Logout:** In Step 6 (GET /logout), LogoutController invoked session.invalidate(). Any subsequent request carrying the old JSESSIONID is rejected by AuthFilter (or assigned a new unauthenticated session when login.jsp renders), confirming server-side state destruction.
- **Zero Password Exposure in Network Response:** In DevTools -> Network -> POST /login, the form data payload transmits username and password in the request body, while the 302 Found response headers and subsequent /dashboard HTML body never echo the password or BCrypt hash.




localhost:8080/student-mvc/login?message=You+have+been+logged+out
Login
Student Management System
You have been logged out
Username
Enter username
Password
Enter password
Remember me
Sign In
Test credentials
Admin: admin / password123
User: john / password123
Elements
Console
Sources
Network
Performance
Memory
Application
Cookie Value
Show URL-decoded
4C4600209337FF19ED1934A2026DABA2


localhost:8080/student-mvc/student?action=list
Student Management System
Welcome, John Doe user Dashboard Logout Light
Student Directory
View, search, sort, and manage all enrolled university students
Export CSV
Dashboard
Search by student code, name, email or ma
Search
Show: 5 per page
Enrolled students database • Total records: 5
ID
STUDENT CODE
STUDENT NAME
EMAIL ADDRESS
MAJOR
#3 ITCSIU21001 Le Van Cuong levanc@student.hcmiu.edu.vn Compute Science
#4 ITDSIU21001 Pham Thi Dung phamthid@student.hcmiu.edu.vn Data Science
#1 ITITIU21001 Nguyen Van An nguyenvanan@student.hcmiu.edu.vn
#2 ITITIU21002 Tran Thi Binh tranthib@student.hcmiu.edu.vn
#5 ITSEIU21001 Hoang Van Em hoangvane@student.hcmiu.edu.vn Software Engineeri
© 2026 EduManage System • Total Enrolled Students: 5
Network
Name
Status
Type
Initiator
Size
Time

Part 2: Section 7, Exercise 1 — Complete Authentication Layer (Basic)

1.1 Task A: Summary of HTTP Status Codes Across the 10 Scenarios (302 Found vs 200 OK)

Inspecting the Network waterfall across Steps 1 through 10 reveals a clear architectural distinction between two HTTP response patterns in a Jakarta EE MVC application: client-side redirection (302 Found) versus server-side forwarding and direct rendering (200 OK).

HTTP Status Pattern	Testing Steps	Originating Component	Technical Mechanism & Architectural Rationale
302 Found -> 200 OK (Redirect)	Steps 1, 3, 6, 7, 8, 9, 10	AuthFilter, AdminFilter, LoginController, LogoutController	Issues HTTP 302 with a Location header, instructing the browser to make a fresh GET request. Applies the Post/Redirect/Get (PRG) pattern in Steps 3 & 7 to prevent duplicate form submission upon browser refresh.
200 OK (Internal Forward)	Step 2 (Failed Login)	LoginController.doPost()	Transfers control internally inside the servlet container to /views/login.jsp without a client round-trip, preserving request-scoped error and username attributes while returning HTTP 200 OK directly.
200 OK (Authorized Render)	Steps 4, 5	StudentController.doGet()	After AuthFilter and AdminFilter pass the request down the filter chain, StudentController queries StudentDAO and forwards directly to student-list.jsp or student-form.jsp with HTTP 200 OK.

1.2 Task B: Server-Side Audit Logging in AdminFilter

In the baseline implementation, AdminFilter silently redirected non-admin users attempting write operations on /student without generating a server-side security record. For Task B, the deny branch of AdminFilter was updated to inspect the active HttpSession, retrieve the authenticated User object's username alongside the blocked request action parameter (new, insert, edit, update, or delete), and write a formatted audit entry to the standard error stream (System.err) with an ISO-8601 local timestamp before sending the HTTP 302 redirect.

- **Audit Log Format Specification:** [AUDIT] <ISO-8601-Timestamp> | UNAUTHORIZED | user=<username> | action=<attempted_action>
- **Verification Outcome (Step 8 Test):** When logged in as regular user john and navigating directly to /student-mvc/student?action=delete&id=1, AdminFilter blocked execution of StudentController, redirected the browser to the student list with the access-denied banner, and printed the following entry to the Tomcat console: [AUDIT] 2026-11-13T19:15:42.318402 | UNAUTHORIZED | user=john | action=delete.

1.3 Task C: Application-Scope Caching of cachedStudentCount in ServletContext

To eliminate redundant SELECT COUNT(*) database queries on every dashboard load, a read-through cache with write-invalidation was implemented using the application-wide ServletContext object:

- **Read-Through Caching in DashboardController:** Upon receiving GET /dashboard, DashboardController looks up the Integer attribute cachedStudentCount in ServletContext. On a cache hit (non-null value), the cached integer is bound directly to the request attribute totalStudents without opening a JDBC connection. On a cache miss (first request after server startup or following invalidation), DashboardController invokes StudentDAO.getTotalStudents(), stores the returned count under cachedStudentCount in ServletContext, and forwards to dashboard.jsp.
- **Targeted Cache Invalidation in StudentController:** Whenever an administrator successfully adds a new student (insert action) or removes an existing student (delete action), StudentController removes the cachedStudentCount attribute from ServletContext immediately after the DAO operation completes. Read-only actions (list, edit form display) and in-place record updates (update action, which does not change the total row count) leave the cached count intact.
- **Thread-Safety & Concurrency Rationale in Apache Tomcat:** In Apache Tomcat 10.1 (ApplicationContext), ServletContext attributes are stored inside an internal ConcurrentHashMap. Consequently, concurrent calls to getAttribute(), setAttribute(), and removeAttribute() across multiple HTTP worker threads are structurally thread-safe and never corrupt the map. Furthermore, because java.lang.Integer is immutable, any benign race on a cold cache miss simply causes two threads to compute the same integer count and store it atomically, while removeAttribute() after an insert or delete guarantees that subsequent dashboard requests fetch fresh data.

1.4 Exercise 1 Written Explanation: Server-Side Session Lifecycle, JSESSIONID Exchange & Invalidation

When a user submits valid credentials via POST /login, LoginController invokes UserDAO.authenticate(username, password). Inside UserDAO, a parameterized PreparedStatement retrieves the active database record matching the submitted username and verifies the plaintext password against the stored BCrypt hash ($2a$10$...) using BCrypt.checkpw(). Once verified, UserDAO populates a User JavaBean containing the user's primary key id, username, hashed password, fullName, role ('admin' or 'user'), isActive status, and audit timestamps, updates the last_login column in MySQL, and returns the User instance to LoginController.

Before binding the authenticated user to a session, LoginController enforces session fixation protection. It inspects whether a pre-login session already exists by calling request.getSession(false); if non-null, it immediately destroys that session via invalidate() so an attacker who pre-seeded a session ID cannot hijack the newly authenticated state. Next, LoginController invokes request.getSession(true). This instructs Tomcat's session manager (StandardManager) to instantiate a new StandardSession object in JVM heap memory, register it in the server's concurrent session table keyed by a cryptographically random 128-bit (32-hex-character) session ID, and configure its inactivity timeout to 1,800 seconds (30 minutes). LoginController then populates the server-side session attribute map with three attributes: user (the User JavaBean reference), role (the role string), and fullName (the display name). Because these attributes remain strictly inside server memory, the client browser never receives the serialized User object. Instead, when LoginController triggers the post-login redirect, Tomcat emits the response header Set-Cookie: JSESSIONID=<new-session-id>; Path=/student-mvc; HttpOnly.

On every subsequent HTTP request to the application (such as GET /dashboard or GET /student?action=list), the browser automatically attaches Cookie: JSESSIONID=<new-session-id> in the request headers. Tomcat's HTTP connector parses this identifier and looks up the corresponding StandardSession instance in StandardManager's in-memory hash table. When AuthFilter, AdminFilter, or a Servlet controller invokes request.getSession(false), Tomcat returns a reference to that exact session object and updates its last-accessed timestamp, allowing the application to read the authenticated User object and enforce role permissions in O(1) time.

Finally, when the user clicks Logout and LogoutController invokes session.invalidate(), two distinct effects occur across the server and the browser. On the server side, Tomcat immediately unbinds all attributes (user, role, fullName), marks the StandardSession instance as invalid, and removes its session ID entry from StandardManager's active lookup table so the object can be garbage-collected. On the browser side, session.invalidate() does not by itself force the browser to delete the local cookie unless an explicit Max-Age=0 header or replacement session cookie is sent. However, even if the browser transmits the stale JSESSIONID value on a subsequent request, Tomcat no longer finds that ID in its active session registry; request.getSession(false) in AuthFilter returns null, and the user is immediately redirected back to /login.

Part 3: Section 7, Exercise 2 — Cookie-Based Theme Preference & Remember Me (Intermediate)

2.1 Task A: ThemeController (/theme) Design & Validation Workflow

To persist a user's visual theme preference across browser sessions without storing non-sensitive UI state in the database, the ThemeController servlet was mapped to /theme with the following request-processing pipeline:

- **Strict Allowlist Parameter Validation:** ThemeController reads the GET query parameter mode and validates it against an explicit server-side allowlist ('light' or 'dark'). Any missing, malformed, or unexpected parameter value is rejected immediately with HTTP 400 Bad Request, preventing arbitrary cookie value injection or HTTP response header manipulation.
- **Persistent Cookie Issuance (365-Day Lifetime):** When a valid mode is supplied, ThemeController delegates to CookieUtil.set() to issue a cookie named user_theme with Max-Age set to 31,536,000 seconds (365 days), Path=/, and HttpOnly=true.
- **Context-Aware Referer Redirection:** After attaching the user_theme cookie to the response, ThemeController inspects the incoming HTTP Referer header. If the Referer header is present and belongs to the application context path, the user is redirected seamlessly back to the exact page they were viewing; otherwise, the controller falls back safely to /dashboard.

2.2 Task B: Theme-Aware JSP Views (dashboard.jsp & student-list.jsp)

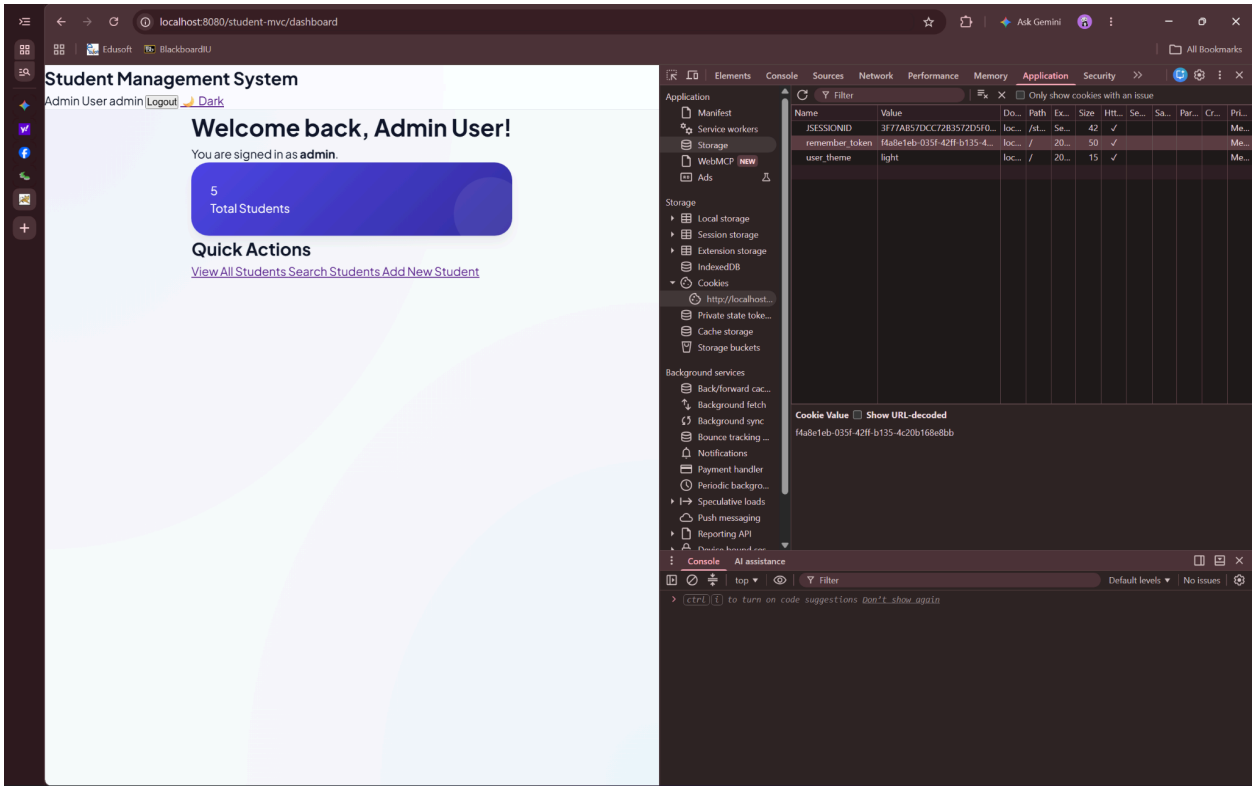
Both primary post-login views—dashboard.jsp and student-list.jsp—were updated to read the user_theme cookie dynamically via JSTL/EL and render a responsive theme toggle control in the top navigation bar:

- **Cookie Resolution via EL Implicit Object:** At the top of each JSP, JSTL evaluates the implicit ${cookie.user_theme.value} expression, defaulting to 'light' whenever the cookie is absent, and computes the opposite mode ('dark' or 'light') for the toggle action.
- **CSS Class Binding & Dark-Palette Overrides:** The resolved theme name is applied to the document body element (theme-light or theme-dark). When theme-dark is active, scoped CSS rules switch the page background to deep slate (#121826), cards and data tables to dark navy (#1E293B), and typography to high-contrast light gray (#F8FAFC).
- **Navigation Bar Toggle Control:** A dedicated 'Switch to Dark/Light Mode' button in the navigation bar links to /theme?mode=<nextTheme>, enabling single-click toggling from either view.

2.3 Task C: Persistent Token-Based Remember Me Architecture

To support automatic re-authentication across browser restarts without exposing credentials or user roles in client storage, a five-part persistent Remember Me mechanism was implemented across MySQL, UserDAO, LoginController, AuthFilter, and LogoutController:

Architectural Layer	Component / Artifact	Implementation Specification & Security Controls
1. Database Schema	remember_tokens Table	Stores id (PK), user_id (FK referencing users.id with ON DELETE CASCADE), token (VARCHAR(255) UNIQUE with B-Tree index idx_token), and expires_at (TIMESTAMP NOT NULL).
2. DAO Persistence	UserDAO PreparedStatement Methods	Implements saveRememberToken(), findUserByValidToken(), and deleteRememberToken() using parameterized PreparedStatements. Token lookup joins users and remember_tokens and enforces rt.expires_at > NOW() AND u.is_active = TRUE.
3. Token Issuance	LoginController.doPost()	When the 'Remember me' checkbox is checked, generates a 128-bit random UUID token, persists it in remember_tokens with a 30-day expiration timestamp, and writes a 30-day HttpOnly cookie named remember_token.
4. Session Restoration	AuthFilter.doFilter()	If no active HttpSession exists, reads the remember_token cookie and queries findUserByValidToken(). If valid, creates a fresh HttpSession populated with user, role, and fullName; if expired or invalid, deletes the stale cookie.
5. Logout Cleanup	LogoutController	Reads remember_token from the request, deletes the token row from MySQL via deleteRememberToken(), expires the browser cookie with Max-Age=0, and invalidates the server-side HttpSession.



2.4 Exercise 2 Written Explanation: Security Analysis of Token-Based Remember Me vs Client-Side Session Cookies

Implementing persistent authentication requires careful evaluation of where state is stored and how trust is established across browser restarts. A naive design stores the entire session state—such as user ID, username, and role—directly inside a long-lived client-side cookie. In contrast, the database-backed Remember Me pattern implemented in Task C stores only an opaque, randomly generated 128-bit UUID token (remember_token) in an HttpOnly cookie while maintaining the authoritative identity mapping and expiration timestamp inside the server-side remember_tokens table.

The attack surfaces of these two approaches differ fundamentally across confidentiality, integrity, and revocation. When full session data is serialized into a long-lived client cookie, any weakness in signing or encryption exposes the application to privilege escalation and permanent account takeover. If the cookie is plain text or weakly encoded (such as Base64), a regular user can trivially modify their role attribute from user to admin in DevTools and gain full administrative privileges. Even if the client-side session cookie is cryptographically signed using HMAC-SHA256 (similar to a stateless JWT), it suffers from two severe structural flaws: first, sensitive metadata (user IDs, roles, account details) is stored on the client and transmitted on every HTTP request; second, stateless client-side cookies cannot be revoked on the server before their 30-day Max-Age expires without re-introducing a server-side blocklist. If a user's password is changed or an administrator deactivates the account (is_active = FALSE), a self-contained cookie continues to grant access until its expiration timestamp passes.

By contrast, an opaque remember_token UUID carries zero intrinsic meaning. It cannot be forged or modified to escalate privileges because privileges are resolved exclusively on the server by joining remember_tokens against the users table and verifying both token expiration and account activity status. Nevertheless, if an attacker steals the remember_token cookie—for example, via network eavesdropping over unencrypted HTTP if the Secure attribute is omitted, physical access to an unattended workstation's browser profile, or local malware extracting the browser cookie store—the attacker can replay the stolen remember_token cookie from their own browser. AuthFilter will match the token in MySQL, construct a valid authenticated HttpSession for that user, and grant full access to the account for up to 30 days without requiring the user's password.

Explicit token invalidation on logout directly narrows this exposure window. When the legitimate user clicks Logout, LogoutController deletes the corresponding row from the remember_tokens table in MySQL in addition to clearing the browser cookie via Max-Age=0. Because the server database is the single source of truth, deleting the database record immediately renders any previously copied remember_token useless—even if the attacker retained a copy of the cookie string.

To further minimize the window of exposure while the user remains logged in, production systems implement rolling token rotation (popularized by Barry Jaspan's persistent login cookie architecture). In a rolling token design, the database table stores both a persistent series identifier (which remains constant for a single login session on a specific device) and a single-use token value. Every time AuthFilter uses remember_token to auto-authenticate a new session, the server immediately invalidates the consumed token value, generates a brand-new random UUID token under the same series identifier, updates the database row, and issues a replacement Set-Cookie header. Consequently, a stolen token is valid only until the legitimate user next opens the application. More importantly, if an attacker uses the stolen token first (causing it to rotate) and the legitimate user subsequently presents the older token from the same series, the server detects that an already-consumed token was replayed for an active series. The server immediately recognizes a token theft event, purges all rows in remember_tokens for that user across all devices, invalidates active sessions, and forces full password re-authentication.

Part 4: Section 7, Exercise 3 — Production Security Hardening (Advanced)

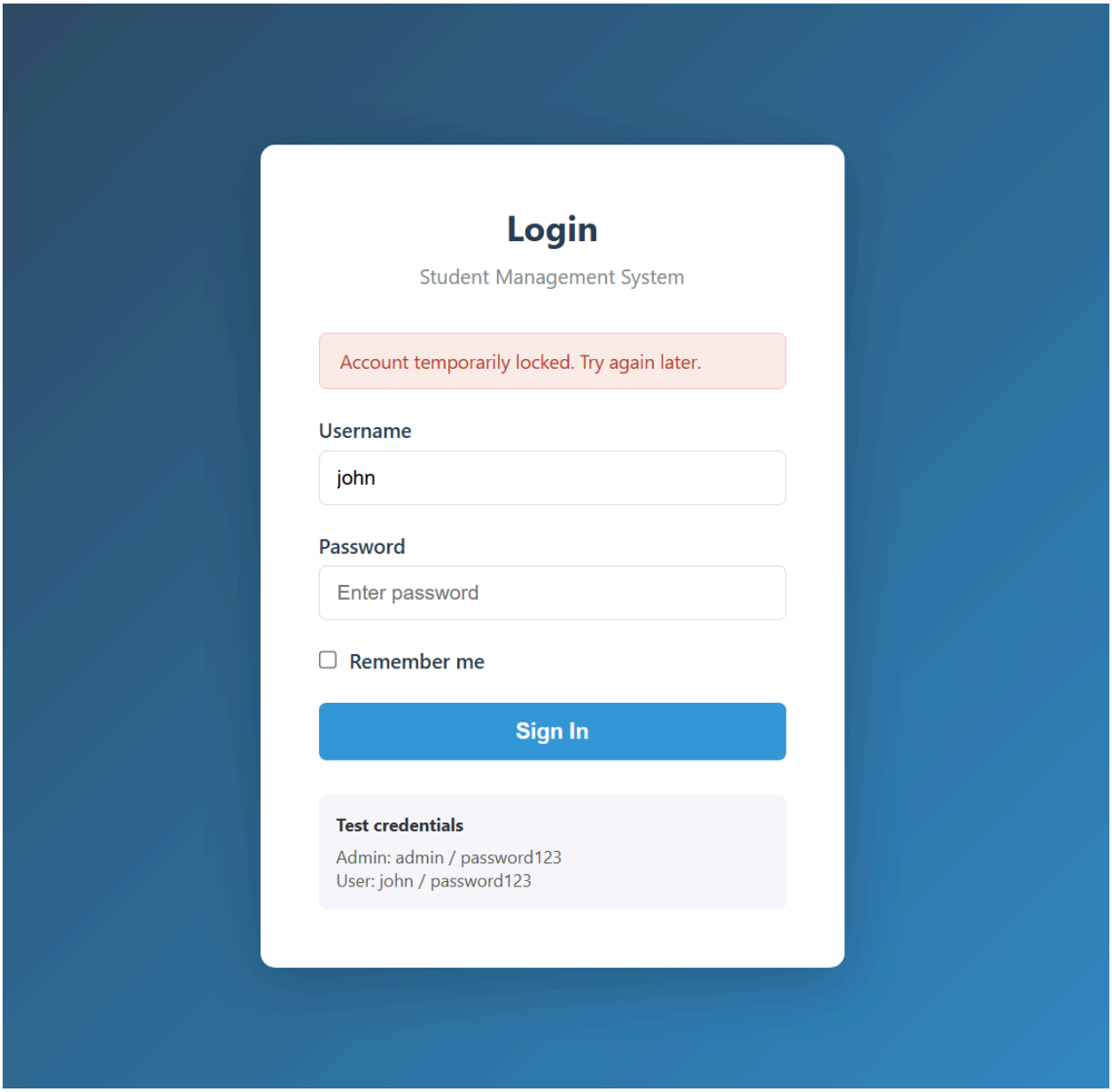
3.1 Task A: Thread-Safe In-Memory LoginAttemptTracker & Distributed Rate Limiting Analysis

To protect the authentication endpoint against automated online brute-force attacks without exhausting database connections or BCrypt CPU threads, an in-memory LoginAttemptTracker utility was implemented and integrated at the entry point of LoginController.doPost():

- **Thread-Safe State Management & Lockout Policy:** LoginAttemptTracker maintains a static ConcurrentHashMap mapping each normalized (trimmed, lowercase) username to an immutable AttemptRecord containing the consecutive failure count, the timestamp of the first failure in the current window, and the lockout expiration timestamp. Using atomic compute() operations on the

ConcurrentHashMap, the fifth consecutive failed login within a 15-minute window sets a 15-minute lockout period. While locked, LoginController short-circuits before calling UserDAO.authenticate() and returns the message 'Account temporarily locked. Try again later.' A successful login prior to lockout invokes reset(username) to remove the tracking entry immediately.

- **Why In-Memory Tracking Fails in Multi-Server Deployments:** While ConcurrentHashMap guarantees thread safety within a single JVM process, it is ineffective in horizontally scaled production environments where multiple Tomcat instances run behind a load balancer (such as Nginx, HAProxy, or AWS ALB). Because each Tomcat server maintains an isolated JVM heap, round-robin load balancing distributes an attacker's requests across N servers—allowing up to (5 * N - 1) failed attempts before any single node locks the account, and allowing requests routed to peer nodes to bypass a lockout entirely. Additionally, container restarts or auto-scaling events wipe in-memory lockout state.
- **Production Architecture (Centralized Redis Cluster):** Production identity providers externalize rate-limiting state into a shared, low-latency distributed cache such as Redis. By executing an atomic INCR and EXPIRE (900 seconds) operation keyed by username and client IP on a centralized Redis cluster, every application node enforces a single unified counter with sub-millisecond latency and automatic TTL expiration.

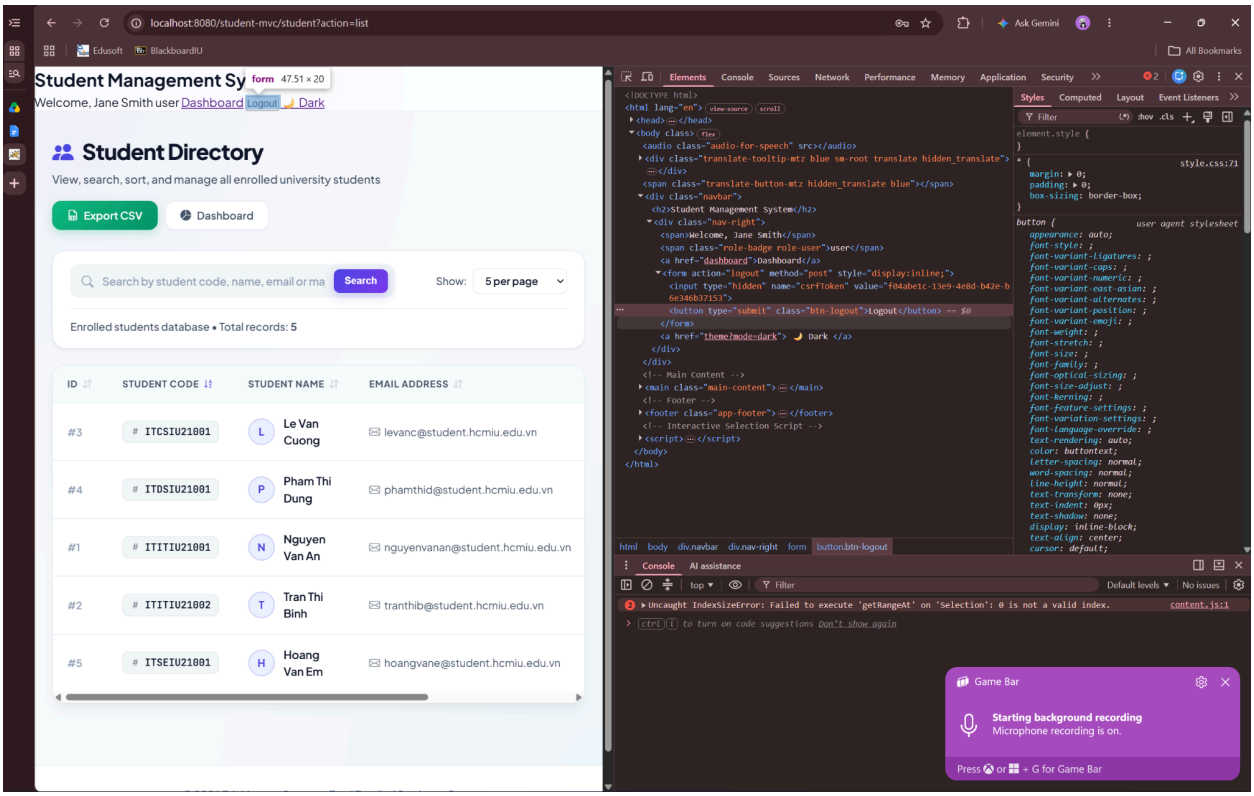


3.2 Task B: Session-Based Synchronizer Token CSRF Protection for Logout

In the baseline application, LogoutController accepted HTTP GET requests (/logout), making it vulnerable to Cross-Site Request Forgery via embedded images or links on third-party websites. Task B hardens the logout workflow using the Synchronizer Token Pattern across three components:

- **CSRF Token Generation on Login:** Upon successful authentication in LoginController (and session restoration in AuthFilter), the server generates a cryptographically random UUID token and stores it in the session under the attribute csrfToken.
- **POST Form Replacement in JSP Views:** In both dashboard.jsp and student-list.jsp, the GET anchor link for Logout was replaced with an inline HTML POST form targeting /logout that embeds the session's csrfToken inside a hidden input field.

- **Method Enforcement & Constant-Time Token Verification in LogoutController:** LogoutController.doGet() was modified to reject GET requests with HTTP 405 Method Not Allowed. In LogoutController.doPost(), the controller retrieves the session's csrfToken and compares it against the submitted form parameter using constant-time byte comparison (MessageDigest.isEqual). If the token is missing or mismatched, the request is aborted with HTTP 403 Forbidden; only verified POST requests proceed to invalidate the session.



3.3 Task C: Relational Audit Logging Architecture (audit_log & AuditLogger)

To establish a permanent, queryable security trail across authentication and data-modification workflows, the audit_log table and AuditLogger DAO component were deployed to record all seven required event categories:

Action Type Constant	Triggering Component & Method	User ID Recorded	Captured Metadata & Event Description
LOGIN_SUCCESS	LoginController.doPost()	Authenticated user.id	Records successful credential verification, username, and client IPv4/IPv6 address.

Action Type Constant	Triggering Component & Method	User ID Recorded	Captured Metadata & Event Description
LOGIN_FAILURE	LoginController.doPost()	NULL (Unauthenticated)	Records failed password verification or locked-account attempt, submitted username, and client IP.
LOGOUT	LogoutController.doPost()	Authenticated user.id	Records CSRF-verified session termination, username, and client IP address.
STUDENT_CREATE	StudentController.insertStudent()	Admin user.id	Records newly created student's name, email, and major alongside admin identity and IP.
STUDENT_UPDATE	StudentController.updateStudent()	Admin user.id	Records target student ID and updated attributes alongside admin identity and IP.
STUDENT_DELETE	StudentController.deleteStudent()	Admin user.id	Records deleted student ID alongside executing admin identity and client IP.
UNAUTHORIZED_ACCESS	AdminFilter.doFilter()	Non-admin user.id	Records blocked privilege escalation attempt (username, target write action, and client IP).

3.4 Exercise 3 Written Explanation: Information Disclosure, CSRF Attack Modeling & Relational Audit Governance

Task A — Information Disclosure & Username Enumeration Defense: In authentication system design, distinguishing between 'Username does not exist' and 'Password is incorrect' constitutes a classic Information Disclosure vulnerability (classified under CWE-204: Observable Response Discrepancy) that enables Username Enumeration. If UserDAO.authenticate() and LoginController returned distinct error messages for each failure mode, an external adversary could supply a dictionary of thousands of candidate usernames (such as student IDs or institutional email prefixes) with a dummy password and harvest a confirmed list of valid accounts simply by observing which requests return 'Password is incorrect'. Armed with a verified list of active usernames, the attacker reduces a two-dimensional credential search space (Username x Password) to a single dimension (Password only), dramatically increasing the efficiency of targeted credential stuffing, spear-phishing, and password spraying attacks.

By returning null from UserDAO.authenticate() for both non-existent accounts and failed BCrypt comparisons—and rendering the identical generic message 'Invalid username or password' while retaining the submitted username in the input field—the application ensures both failure paths produce indistinguishable HTTP responses. Furthermore, in high-security deployments, developers also mitigate timing side-channels (where a missing username returns in ~1 ms from MySQL while an existing username takes ~100 ms in BCrypt.checkpw()) by executing a dummy BCrypt comparison against a precomputed constant hash whenever the username lookup returns no rows.

Task B — Concrete CSRF Attack Scenario & Synchronizer Token Mitigation: Cross-Site Request Forgery (CSRF) exploits the fact that web browsers automatically attach ambient credentials—specifically the JSESSIONID and remember_token cookies—to every HTTP request directed at an origin, regardless of which website initiated the request. In the initial Lab 06 implementation, LogoutController processed logout requests over HTTP GET (/student-mvc/logout) and verified only that a session existed. Consider a concrete attack scenario: Administrator Phan Nhật Anh logs into http://localhost:8080/student-mvc/ and receives a valid JSESSIONID cookie. While keeping that tab open to edit student records, the administrator opens a second browser tab to view a public discussion forum or HTML email controlled by an attacker. The attacker embeds an invisible image element in that external page pointing to http://localhost:8080/student-mvc/logout. The moment the administrator's browser parses the external page, it automatically issues GET /student-mvc/logout and attaches the administrator's active JSESSIONID cookie. Because LogoutController cannot distinguish this cross-origin image fetch from an intentional click on the navigation bar Logout link, it immediately invalidates the session, terminating the administrator's workflow mid-task and causing unsaved changes in the student form to be lost. Converting the logout action into a POST form containing a hidden session-bound synchronizer token (csrfToken) and rejecting GET requests with HTTP 405 Method Not Allowed completely neutralizes this attack. Although a malicious third-party page can still trigger a cross-origin POST via JavaScript, the browser's Same-Origin Policy (SOP) strictly prevents the attacker's script from reading the victim's HTML DOM or extracting the unpredictable UUID stored in the hidden csrfToken field. When the forged POST arrives without a matching token, LogoutController aborts with HTTP 403 Forbidden.

Task C — Operational Advantages of Database Audit Logs & Retention Policy: Persisting security events in a relational audit_log table provides critical operational, forensic, and compliance capabilities that unstructured server console logs (such as Tomcat's catalina.out or localhost.log) cannot satisfy. Standard application logs interleave stack traces, framework startup messages, and debug output into flat text files stored on local disk. In contrast, a relational audit_log table enforces a strict schema (user_id, action_type, description, ip_address, created_at) with B-Tree indexes that allow security analysts to execute instant SQL queries across dimensions—for example, correlating all STUDENT_DELETE actions performed by a specific user ID over a 30-day window, grouping LOGIN_FAILURE spikes by IP address to detect distributed brute-force botnets, or powering an in-app administrative security dashboard. Moreover, centralized database audit records survive container replacement in ephemeral Docker/Kubernetes environments where local container log files are destroyed on pod termination, and foreign-key constraints (ON DELETE SET NULL) preserve historical accountability even if a user account is later deleted.

Regarding retention duration, security and academic governance frameworks (including ISO/IEC 27001, NIST SP 800-92, and PCI-DSS Requirement 10.7) recommend a tiered retention policy of at least 1 year (365 days), with a minimum of 90 days kept immediately queryable in 'hot' relational storage and months 4 through 12 (or up to 3–7 years for regulated academic transcript and financial systems) archived in append-only, tamper-evident 'cold' storage (such as WORM object storage). Retaining audit records for at least one full year is essential because the industry median dwell time—the interval between an initial credential compromise and its discovery during an audit or academic grade review—frequently spans several months; premature log rotation (such as a 7-day rolling file policy) destroys the forensic trail required to reconstruct root cause and scope of impact.